View.py

```
import tkinter as tk
from tkinter import messagebox, ttk

class CarDatabaseView:
    def __init__(self, root, controller):
        self.root = root
        self.controller = controller
        self.create_widgets()

    def create_widgets(self):
        # Create the treeview for displaying records
        self.tree = ttk.Treeview(self.root, columns=('ID', 'Brand', 'Model', 'Color', 'Year', 'Price',
'Condition'), show='headings')
        for col in self.tree['columns']:
            self.tree.heading(col, text=col)
        self.tree.pack(expand=True, fill=tk.BOTH)

        form_frame = tk.Frame(self.root)
        form_frame.pack(pady=10)

        labels = ['Brand', 'Model', 'Color', 'Year', 'Price', 'Condition']
        self.entries = {}
        self.combobox_values = {
            'brand': ['Toyota', 'Honda', 'Ford', 'Chevrolet', 'Nissan'],
            'color': ['Red', 'Blue', 'Green', 'Black', 'White']
        }
        for i, label in enumerate(labels):
            tk.Label(form_frame, text=label).grid(row=i, column=0, padx=5, pady=5)
            if label.lower() in self.combobox_values:
                entry = ttk.Combobox(form_frame, values=self.combobox_values[label.lower()])
            else:
                entry = tk.Entry(form_frame)
            if label == 'Condition':
                self.voice_recognition_button = tk.Button(form_frame, text="Speak Condition",
command=self.controller.recognize_condition)
```

```
                self.voice_recognition_button.grid(row=i, column=2, padx=5, pady=5)
            entry.grid(row=i, column=1, padx=5, pady=5)
            self.entries[label.lower()] = entry

        # Create action buttons
        self.add_button = tk.Button(self.root, text="Add", command=self.controller.add_record)
        self.add_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.edit_button = tk.Button(self.root, text="Edit", command=self.controller.edit_record)
        self.edit_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.delete_button = tk.Button(self.root, text="Delete", command=self.controller.delete_record)
        self.delete_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.search_button = tk.Button(self.root, text="Search", command=self.controller.search_record)
        self.search_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.sort_brand_button = tk.Button(self.root, text="Sort by Brand",
command=self.controller.sort_by_brand)
        self.sort_brand_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.sort_price_button = tk.Button(self.root, text="Sort by Price",
command=self.controller.sort_by_price)
        self.sort_price_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.sort_year_button = tk.Button(self.root, text="Sort by Year",
command=self.controller.sort_by_year)
        self.sort_year_button.pack(side=tk.LEFT, padx=5, pady=5)

        self.statistics_button = tk.Button(self.root, text="Statistics",
command=self.controller.show_statistics)
        self.statistics_button.pack(side=tk.LEFT, padx=5, pady=5)

    def get_entries_data(self):
        # Retrieve data from entries
        return [self.entries[field].get() for field in self.entries]
```

```
    def clear_entries(self):
        # Clear all entries
        for entry in self.entries.values():
            entry.delete(0, tk.END)

    def insert_tree_data(self, records):
        # Insert data into treeview
        for row in self.tree.get_children():
            self.tree.delete(row)
        for record in records:
            self.tree.insert("", "end", values=record)
```

3

Model.py

```
import sqlite3

class CarDatabaseModel:
    def __init__(self):
        self.conn = sqlite3.connect('cars.db')
        self.create_table()

    def create_table(self):
        with self.conn:
            # Create the table if it doesn't exist
            self.conn.execute(
                '''
                CREATE TABLE IF NOT EXISTS cars (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    brand TEXT,
                    model TEXT,
                    color TEXT,
                    year INTEGER,
                    price REAL,
                    condition TEXT
                )
                '''
            )

    def add_record(self, brand, model, color, year, price, condition):
        with self.conn:
            # Add a new record to the database
            self.conn.execute(
                "INSERT INTO cars (brand, model, color, year, price, condition) VALUES (?, ?, ?, ?, ?, ?)",
                (brand, model, color, year, price, condition)
            )

    def delete_record(self, record_id):
        with self.conn:
            # Delete a record from the database by ID
```

```
        self.conn.execute("DELETE FROM cars WHERE id = ?", (record_id,))

    def update_record(self, record_id, brand, model, color, year, price, condition):
        with self.conn:
            # Update a record in the database by ID
            self.conn.execute(
                '''
                UPDATE cars
                SET brand = ?, model = ?, color = ?, year = ?, price = ?, condition = ?
                WHERE id = ?
                ''',
                (brand, model, color, year, price, condition, record_id)
            )

    def search_records(self, **kwargs):
        query = "SELECT * FROM cars WHERE "
        query += " AND ".join([f"{k} LIKE ?" for k in kwargs.keys()])
        values = [f"%{v}%" for v in kwargs.values()]
        # Search for records based on criteria
        cursor = self.conn.execute(query, values)
        return cursor.fetchall()

    def get_all_records(self):
        # Get all records from the database
        cursor = self.conn.execute("SELECT * FROM cars")
        return cursor.fetchall()

    def get_car_counts_by_brand(self):
        # Get count of cars grouped by brand
        cursor = self.conn.execute("SELECT brand, COUNT(*) FROM cars GROUP BY brand")
        return cursor.fetchall()

    def sort_by(self, field):
        # Sort records by a specific field
        cursor = self.conn.execute(f"SELECT * FROM cars ORDER BY {field}")
        return cursor.fetchall()
```

5

Controller.py

```
import tkinter as tk
from tkinter import messagebox
from view import CarDatabaseView
from model import CarDatabaseModel
import speech_recognition as sr
import sounddevice as sd
import numpy as np
import io
from bokeh.plotting import figure, show, output_file
from bokeh.models import ColumnDataSource

class CarDatabaseController:
    def __init__(self, root):
        self.model = CarDatabaseModel()
        self.view = CarDatabaseView(root, self)
        self.load_data()

    def load_data(self):
        # Load all records into the view
        records = self.model.get_all_records()
        self.view.insert_tree_data(records)

    def validate_entry(self, data):
        # Validate entry data
        try:
            year = int(data[3])
            price = float(data[4])
        except ValueError:
            return False
        return all(data[:3]) and all(data[5:]) and year > 0 and price > 0

    def add_record(self):
        # Add a new record
        data = self.view.get_entries_data()
        if self.validate_entry(data):
```

6

```
            self.model.add_record(*data)
            self.load_data()
            self.view.clear_entries()
            messagebox.showinfo("Success", "Record added successfully")
        else:
            messagebox.showerror("Validation Error", "Please provide valid data for all fields")

    def edit_record(self):
        # Edit an existing record
        selected_item = self.view.tree.focus()
        if selected_item:
            record_id = self.view.tree.item(selected_item)['values'][0]
            data = self.view.get_entries_data()
            if self.validate_entry(data):
                self.model.update_record(record_id, *data)
                self.load_data()
                self.view.clear_entries()
                messagebox.showinfo("Success", "Record updated successfully")
            else:
                messagebox.showerror("Validation Error", "Please provide valid data for all fields")
        else:
            messagebox.showerror("Selection Error", "Please select a record to edit")

    def delete_record(self):
        # Delete an existing record
        selected_item = self.view.tree.focus()
        if selected_item:
            record_id = self.view.tree.item(selected_item)['values'][0]
            self.model.delete_record(record_id)
            self.load_data()
            self.view.clear_entries()
            messagebox.showinfo("Success", "Record deleted successfully")
        else:
            messagebox.showerror("Selection Error", "Please select a record to delete")

    def search_record(self):
        # Search for records
```

```
        search_data = self.view.get_entries_data()
        search_dict = {k: v for k, v in zip(self.view.entries.keys(), search_data) if v}
        if search_dict:
            records = self.model.search_records(**search_dict)
            self.view.insert_tree_data(records)
        else:
            messagebox.showerror("Search Error", "Please enter search criteria")

    def sort_by_brand(self):
        # Sort records by brand
        records = self.model.sort_by('brand')
        self.view.insert_tree_data(records)

    def sort_by_price(self):
        # Sort records by price
        records = self.model.sort_by('price')
        self.view.insert_tree_data(records)

    def sort_by_year(self):
        # Sort records by year
        records = self.model.sort_by('year')
        self.view.insert_tree_data(records)

    def recognize_condition(self):
        # Use voice recognition to fill the condition field
        recognizer = sr.Recognizer()
        with sd.InputStream(samplerate=16000, channels=1, dtype='int16') as stream:
            print("Listening...")
            audio = stream.read(int(16000 * 5))  # Record 5 seconds of audio
            audio_data = np.frombuffer(audio[0], dtype=np.int16)
            audio_file = io.BytesIO(audio_data.tobytes())
            audio_data = sr.AudioData(audio_file.getvalue(), 16000, 2)
            try:
                text = recognizer.recognize_google(audio_data)
                print(f"You said: {text}")
                self.view.entries['condition'].delete(0, tk.END)  # Clear the text field
                self.view.entries['condition'].insert(0, text)    # Insert recognized text
```

8

```
            except sr.UnknownValueError:
                print("Sorry, I did not understand that.")
            except sr.RequestError:
                print("Sorry, there was an error with the speech recognition service.")

    def show_statistics(self):
        # Show statistics using Bokeh
        data = self.model.get_car_counts_by_brand()
        if data:
            brands, counts = zip(*data)
            source = ColumnDataSource(data=dict(brands=brands, counts=counts))
            p = figure(x_range=brands, height=350, title="Car Counts by Brand", toolbar_location=None,
tools="")
            p.vbar(x='brands', top='counts', width=0.9, source=source)
            p.xgrid.grid_line_color = None
            p.y_range.start = 0
            output_file("car_counts.html")
            show(p)
        else:
            messagebox.showinfo("Statistics", "No data available for statistics.")
```

Main.py

```
import tkinter as tk
from controller import CarDatabaseController

if __name__ == "__main__":
    root = tk.Tk()
    app = CarDatabaseController(root)
    root.mainloop()
```

10